MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
NATIONAL TECHNICAL UNIVERSITY
“KHARKIV POLYTECHNIC INSTITUTE”
DEPARTMENT OF SOFTWARE ENGINEERING AND MANAGEMENT INFORMATION TECHNOLOGIES

Report of laboratory work № 3

Discipline «Software Engineering»
Theme: Planning of System Activities

Executed by:
Student of group KN-226e
Budkov O. Yu.

Checked by:
Associate professor
Melnyk K. V.

Kharkiv
2026

Theme: planning of system activities.

Goal: to obtain basic experience and knowledge to work with a project tracking system.

Tasks:

– register on the free github.com account;

– create a new repository;

– add a new user as a collaborator to the repository;

– create milestones and issues according to the previous labs and assign tasks to the users of the team;

– work with the git repository;

– prepare the report of the work.

Progress of work

Task 3.1 Registering on github.com

GitHub is a web platform for version control and teamwork. It stores the repository of the project with the full history of changes and gives tools for planning the work: milestones, issues, labels and assignees. A free account was registered on https://github.com/signup (Figure 3.1).

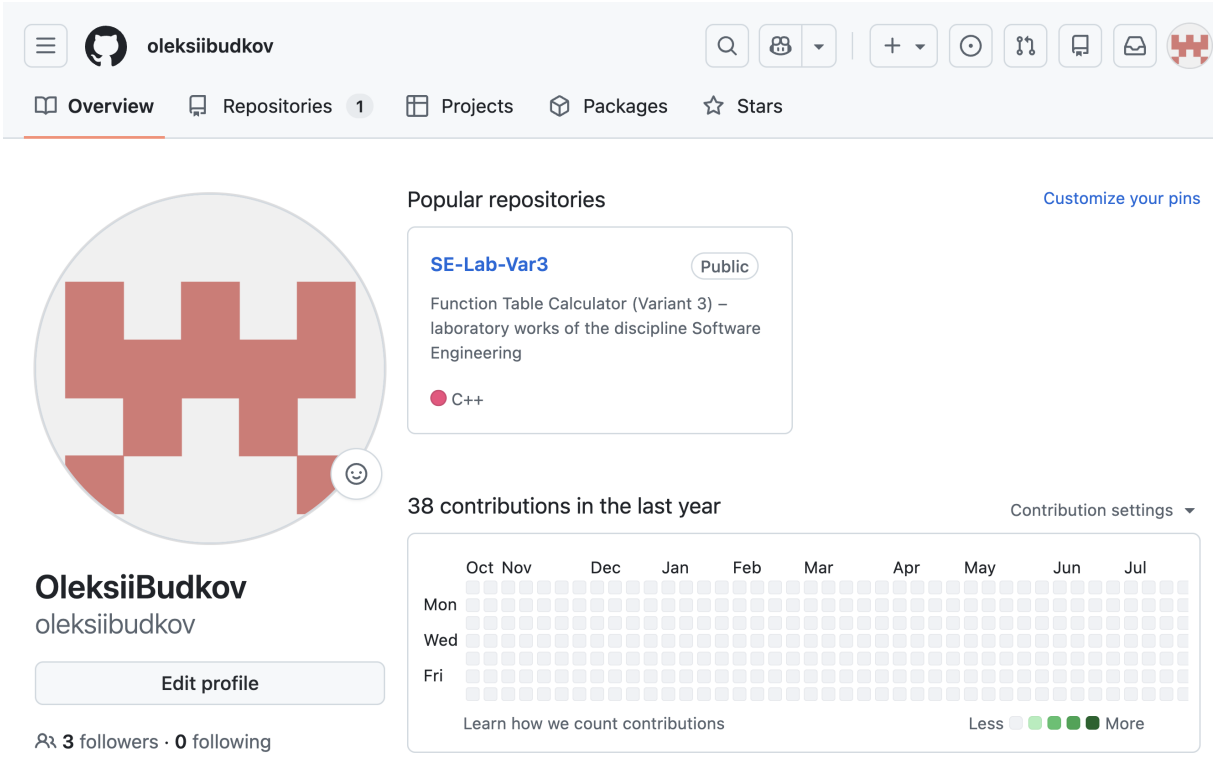


Figure 3.1 – Account on github.com

Task 3.2 Creating a New Repository

A new public repository SE-Lab-Var3 was created with the button “New” (Figure 3.2). Its description is “Function Table Calculator (Variant 3) – laboratory works of the discipline Software Engineering”. The repository stores the C++ program, the reports and the diagrams of all labs (Figure 3.3); the file README.md describes the program and the structure of the repository.

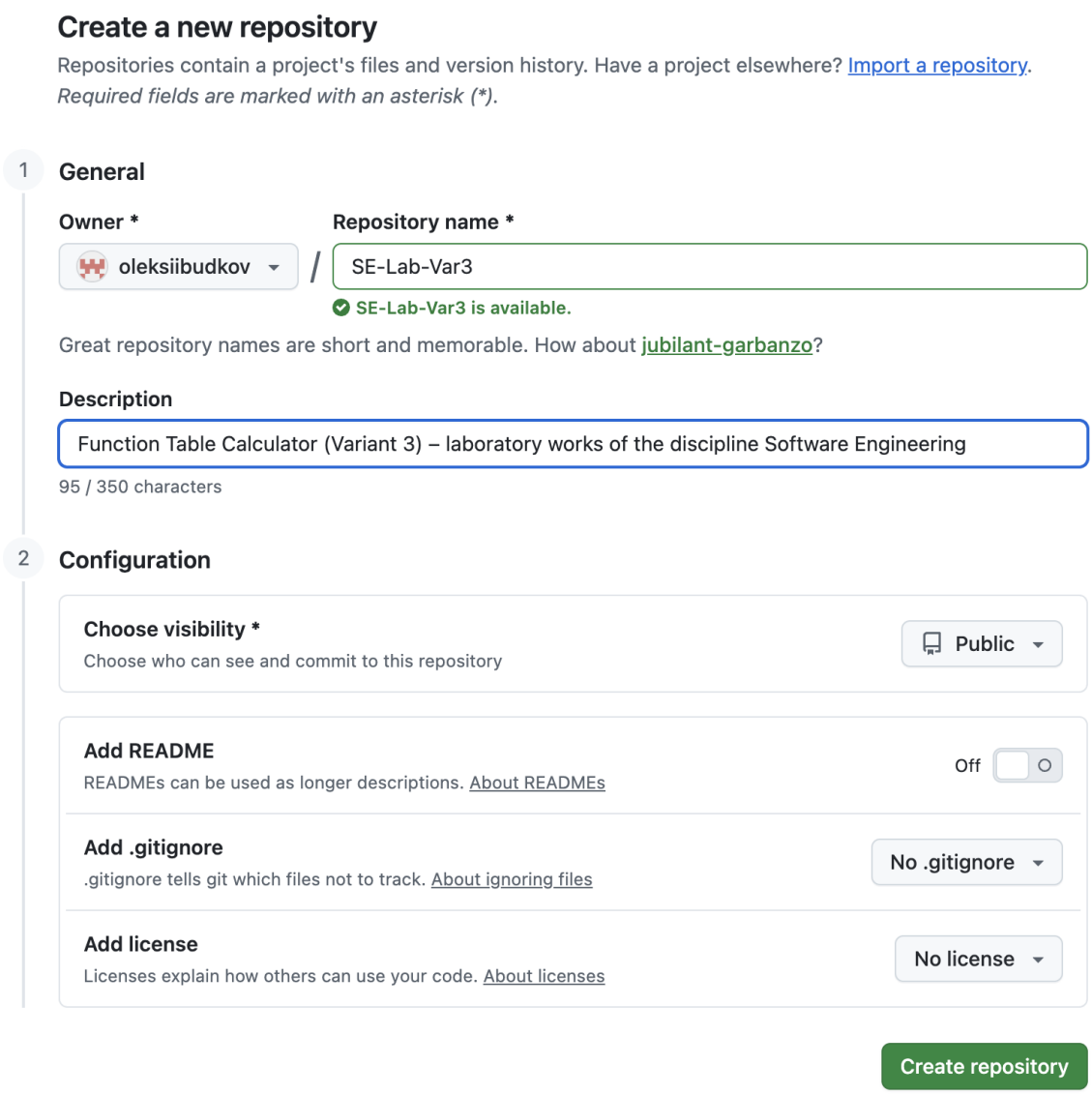


Figure 3.2 – Creating of a new repository

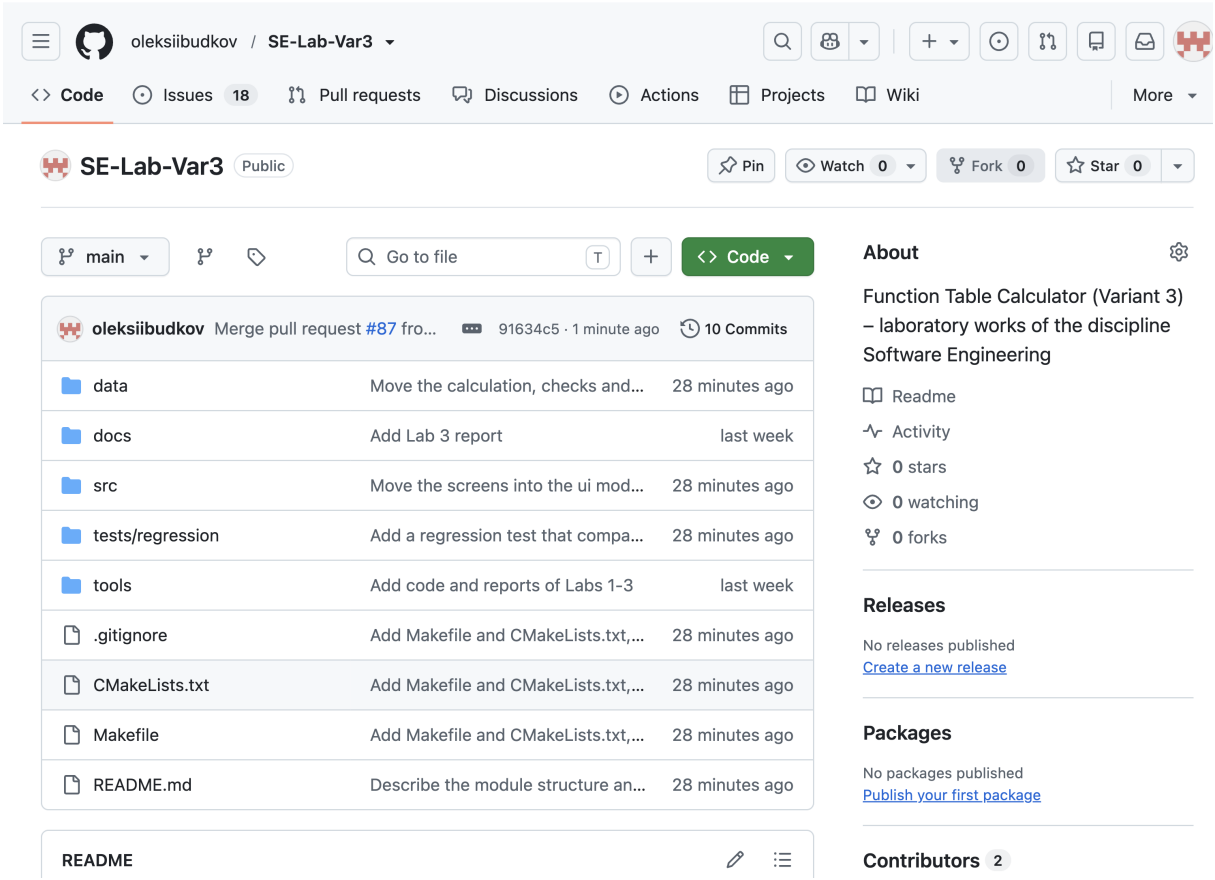


Figure 3.3 – The new repository on github.com

Task 3.3 Adding a Collaborator

The project is developed by one person, so no second member was invited. A collaborator is invited on the page Settings → Collaborators → Add people (Figure 3.4): after the invitation is accepted, the collaborator can push changes to the repository and can be assigned to issues. All issues of the project are assigned to the author of the repository.

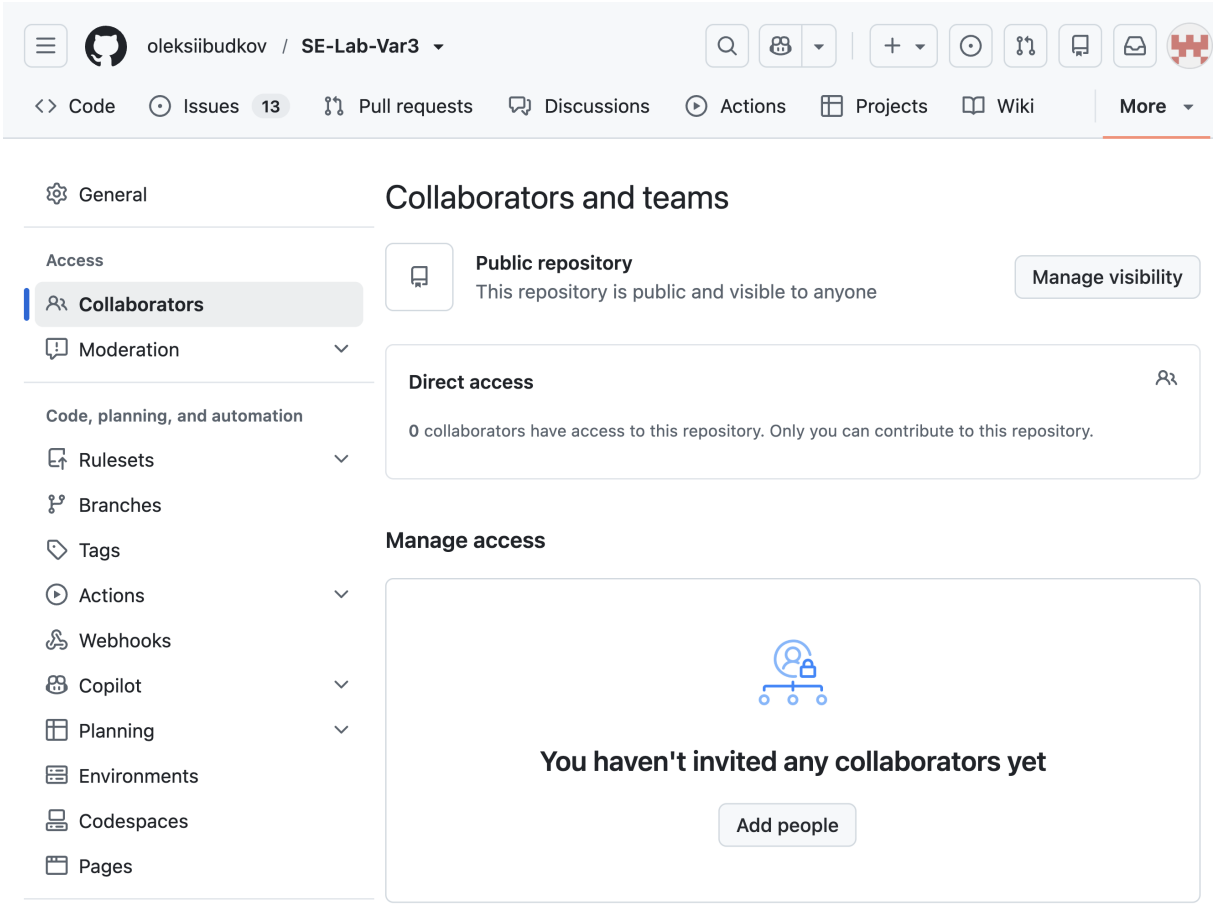


Figure 3.4 – Collaborators of the repository

Task 3.4 Planning with Scrum: Milestones and Issues

The work is planned with Scrum. Instead of a plan of phases (analysis, coding, specification, testing), the project is a sequence of sprints of the same length — three weeks. Every sprint has a sprint goal and ends with an increment of the product that can be shown to the customer. All requirements are kept in the Product Backlog ordered by priority, and the work chosen for the current sprint is the Sprint Backlog. The project is made by one student, so one person has all three Scrum roles: Product Owner, Scrum Master and Developer.

Every Product Backlog item is a user story written as “As a <role>, I want <goal>, so that <benefit>”. A story has acceptance criteria, an estimate in story points (the Fibonacci numbers 1, 2, 3, 5, 8) and a MoSCoW priority: Must, Should or Could have. Labels show the type of an item — user story or task — and its priority (Figure 3.5). A story is created as an issue with these labels, the sprint (milestone) and the assignee (Figure 3.6). The technical work of a story is split into tasks; every task is a sub-issue of its story, so the story shows the progress of its tasks (Figure 3.7).

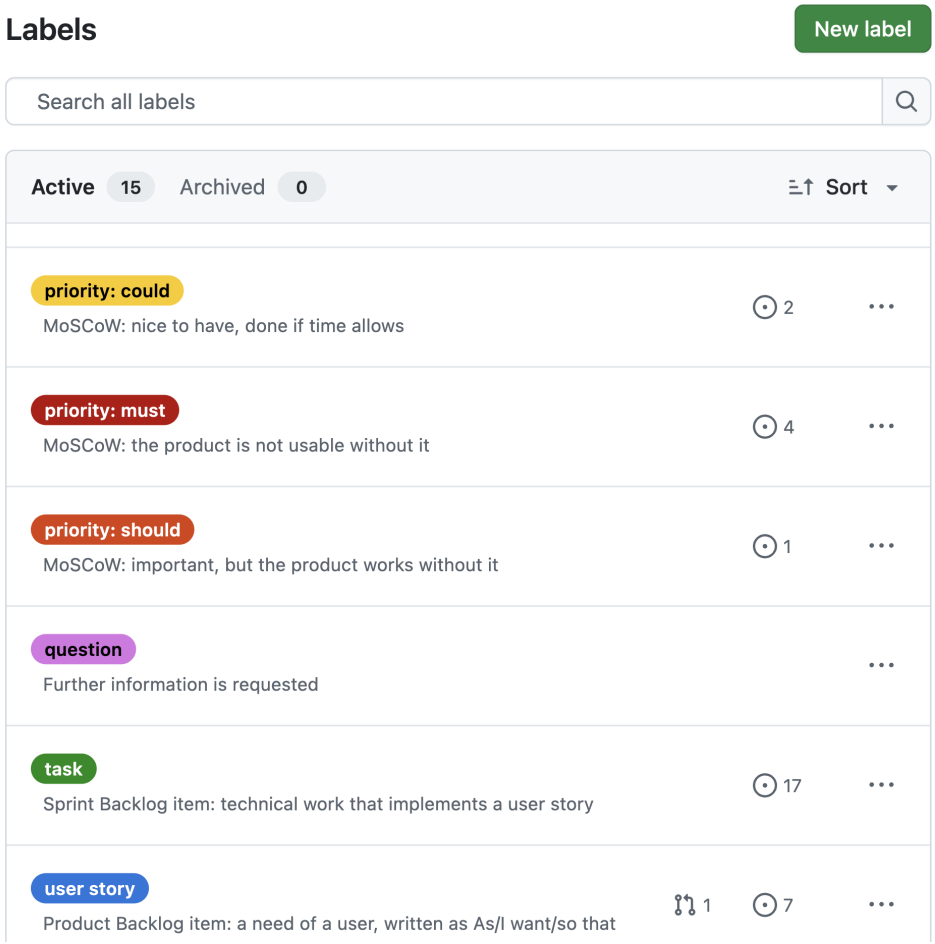


Figure 3.5 – Labels of the Product Backlog items

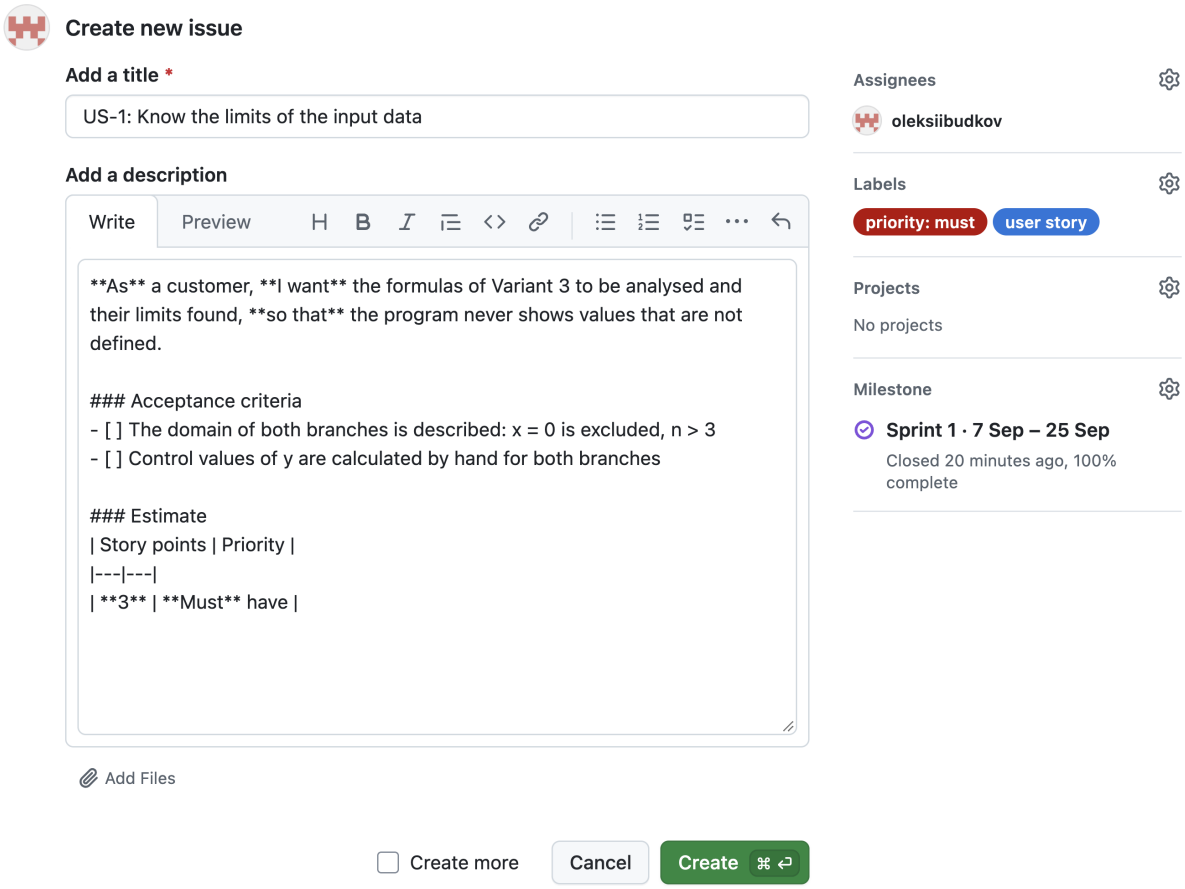


Figure 3.6 – Creating of a user story

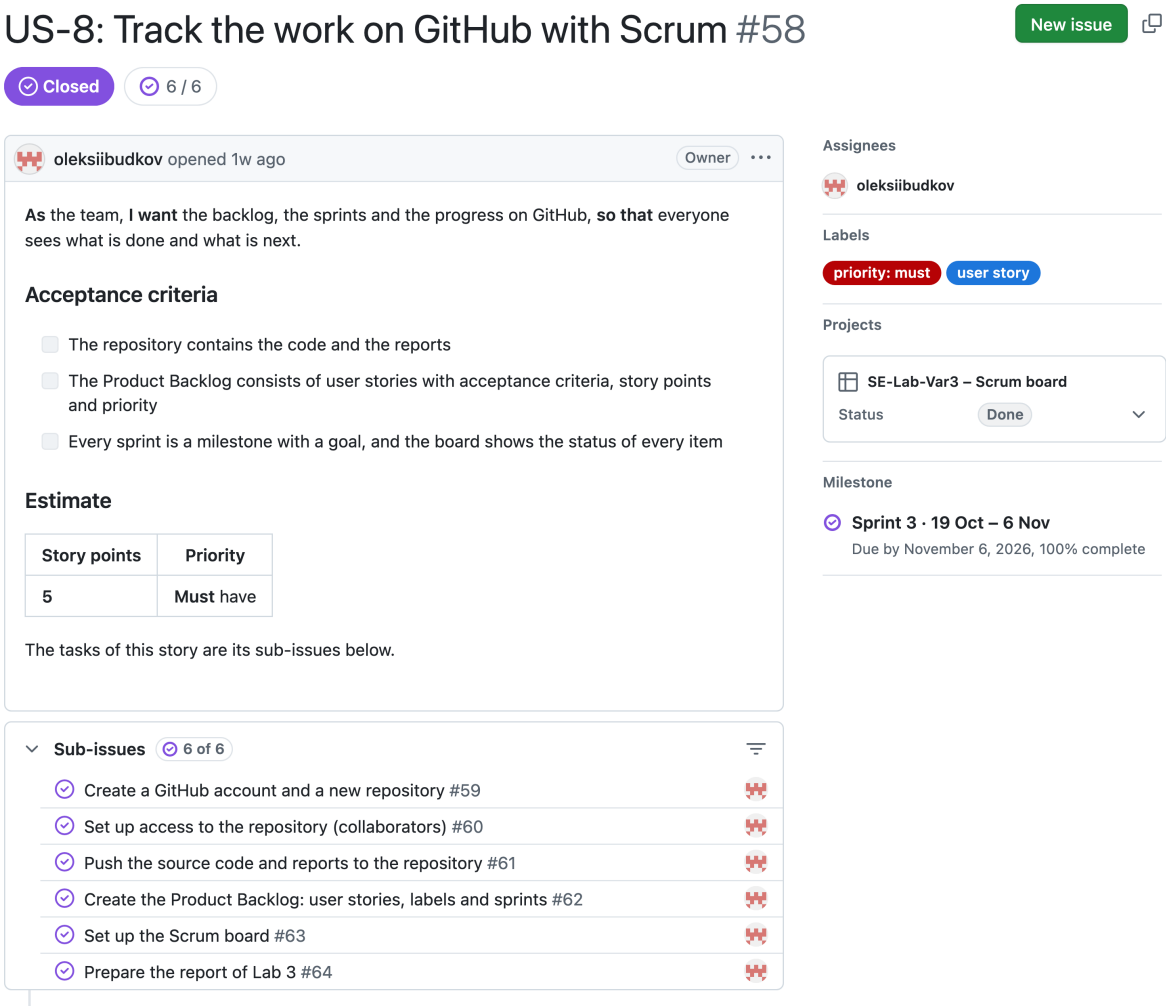


Figure 3.7 – User story with its tasks (sub-issues)

Every sprint is a milestone: the title has the dates of the sprint, the description has the sprint goal, and the due date is the last day of the sprint (Figure 3.8). Five sprints cover the semester (Figure 3.9); the deadlines of the laboratory works fall inside them. When a sprint ends, its milestone is closed and its description keeps the velocity — the story points done — and the main idea of the retrospective. Two stories that are not planned for a sprint yet stay in the Product Backlog.

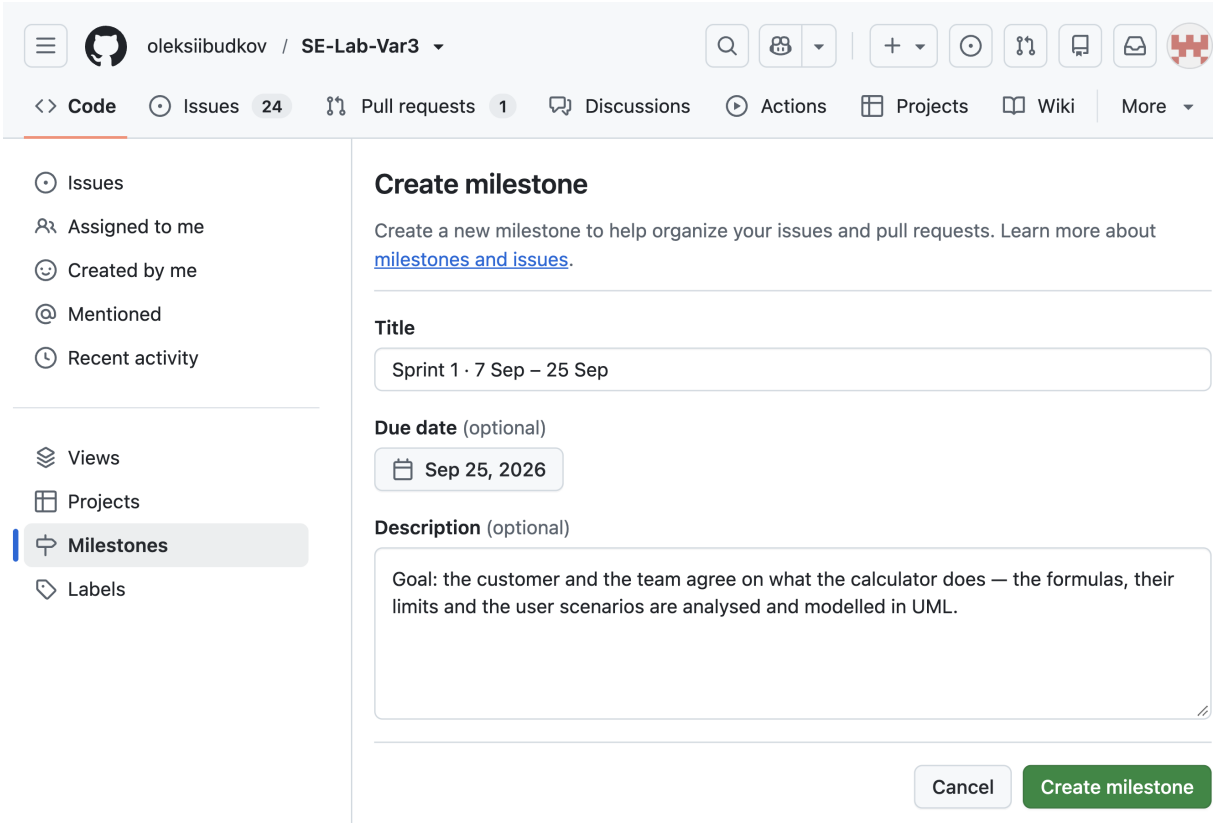


Figure 3.8 – Creating of a sprint (milestone with a sprint goal)

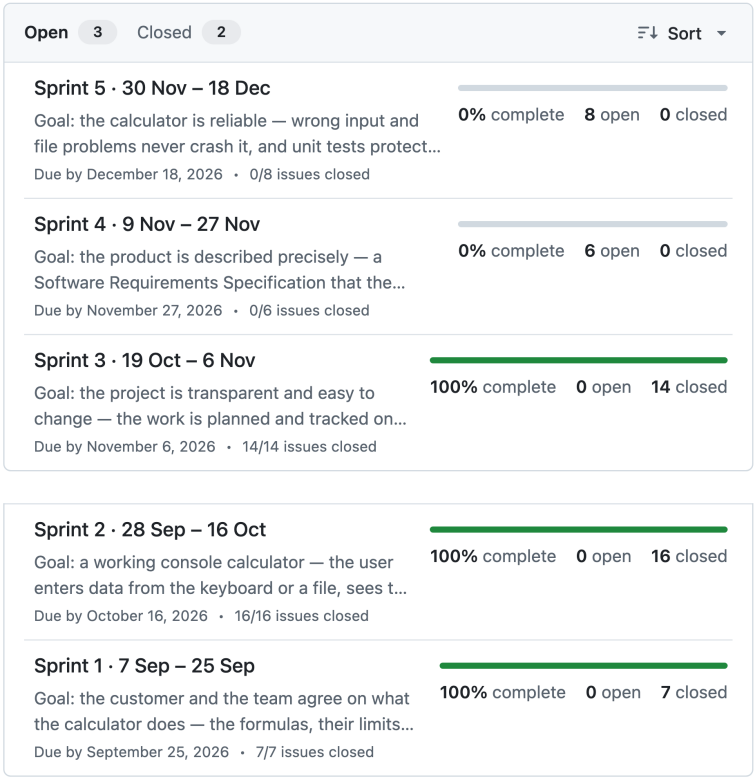


Figure 3.9 – Sprints of the project

An item is done when it meets the Definition of Done: its acceptance criteria are met, the code builds without warnings, the regression test passes, the change is merged into the main branch through a pull request and the documentation is updated. Sprints 1 and 2 are finished: 26 story points, all their stories and tasks are closed, so the milestones show 100 % (Figure 3.10).

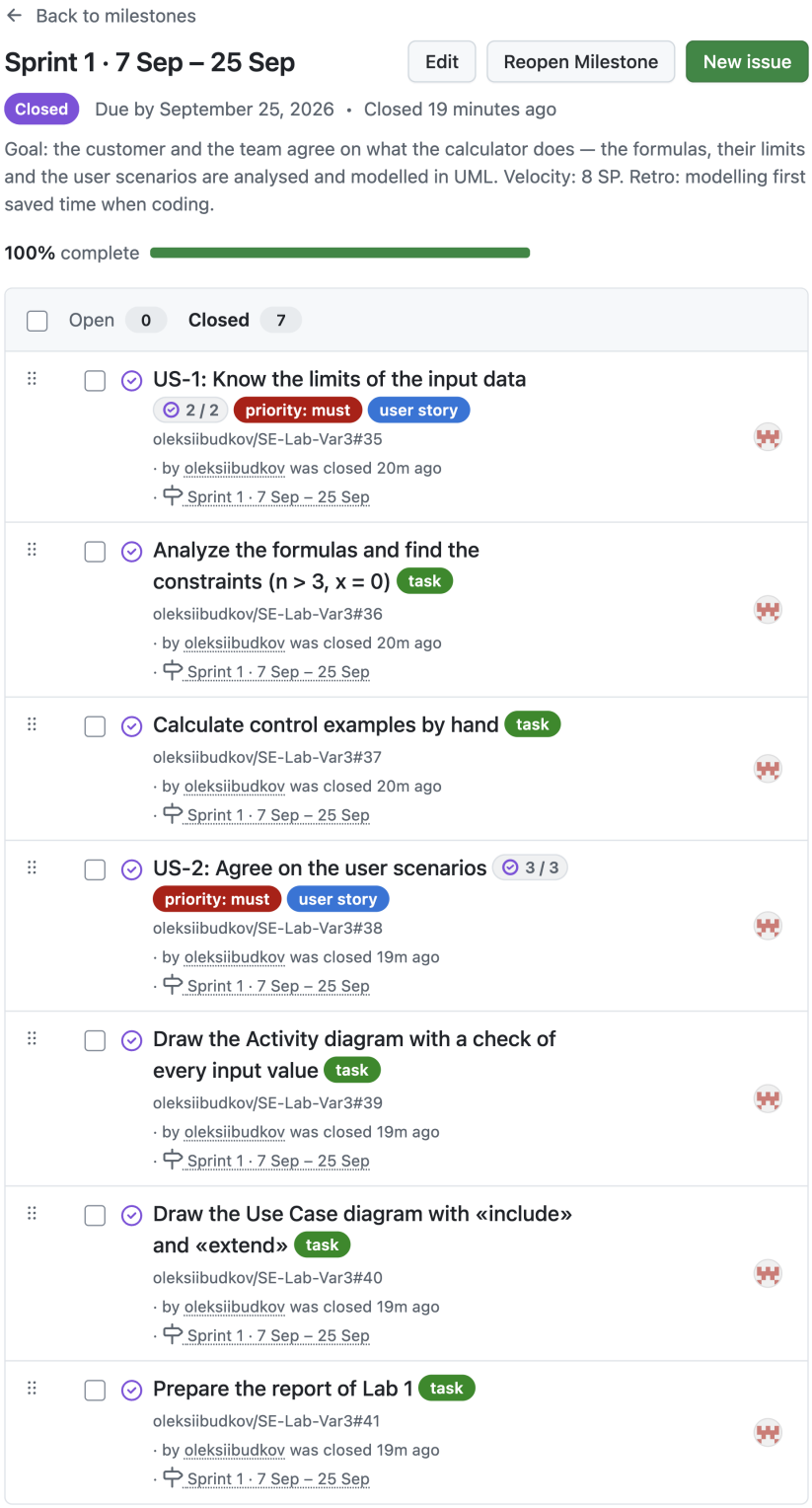


Figure 3.10 – Sprint 1 with its finished stories and tasks

The Scrum board is a GitHub Project linked to the repository. Its field Status has the columns Product Backlog, Sprint Backlog, In Progress and Done, and the fields Story Points and Priority hold the estimate. The view “Product Backlog” groups the stories by sprints and sums their story points (Figure 3.11); the view “Sprint 3 board” shows the current sprint as columns (Figure 3.12). The labels, sprints, stories, tasks and the board are created by the script tools/github_setup.sh with GitHub CLI, so the structure can be built again with one command.

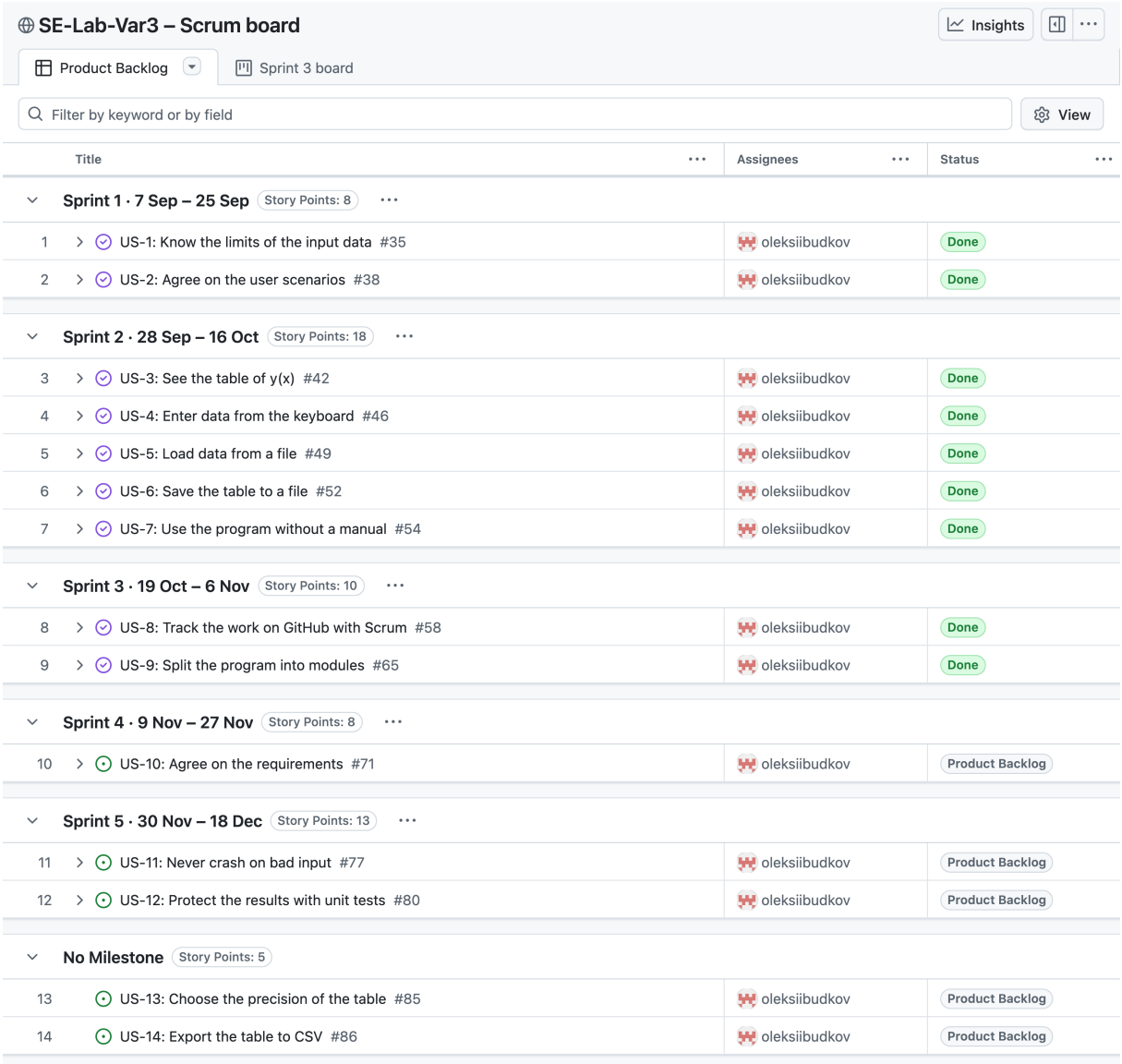


Figure 3.11 – Product Backlog view of the Scrum board

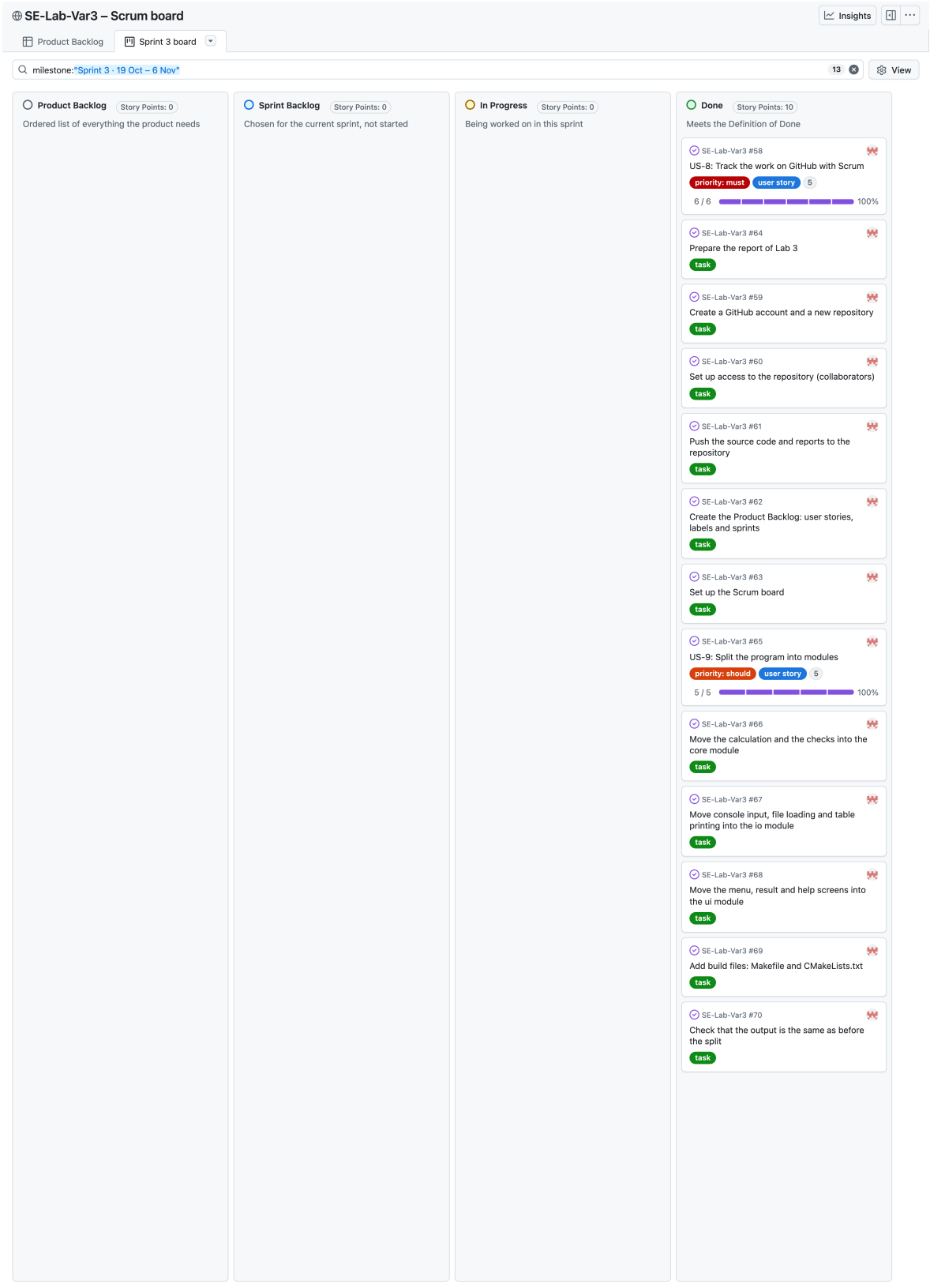


Figure 3.12 – Board of the current sprint

The stories and tasks of every sprint are shown in Tables 3.1–3.5, and the items that are not planned yet — in Table 3.6. The page of every user story on GitHub, with its acceptance criteria, estimate, status and tasks, is shown in Appendix A.

Table 3.1 – Sprint 1, 7.09–25.09.2026. Goal: the behaviour of the calculator is agreed and modelled (8 story points)

User story	Task	Assignee	Status
US-1: Know the limits of the input data (#35) 3 SP, Must have	Analyze the formulas and find the constraints (n > 3, x = 0) (#36)	Budkov O.	Done
	Calculate control examples by hand (#37)	Budkov O.	Done
US-2: Agree on the user scenarios (#38) 5 SP, Must have	Draw the Activity diagram with a check of every input value (#39)	Budkov O.	Done
	Draw the Use Case diagram with «include» and «extend» (#40)	Budkov O.	Done
	Prepare the report of Lab 1 (#41)	Budkov O.	Done

Table 3.2 – Sprint 2, 28.09–16.10.2026. Goal: a working console calculator (18 story points)

User story	Task	Assignee	Status
US-3: See the table of y(x) (#42) 5 SP, Must have	Implement the calculation of y for both branches (#43)	Budkov O.	Done
	Print the result table with “not defined” and “too large” (#44)	Budkov O.	Done
	Compare the results with the control examples (#45)	Budkov O.	Done
US-4: Enter data from the keyboard (#46) 5 SP, Must have	Implement keyboard input with a separate check of every value (#47)	Budkov O.	Done
	Write clear error messages with examples (#48)	Budkov O.	Done
US-5: Load data from a file (#49) 3 SP, Should have	Implement loading of data from a file (#50)	Budkov O.	Done
	Keep the previous data when the file is wrong (#51)	Budkov O.	Done
US-6: Save the table to a file (#52) 2 SP, Should have	Implement saving of the table to results.txt (#53)	Budkov O.	Done
US-7: Use the program without a manual (#54) 3 SP, Should have	Add the main menu and the Help screen (#55)	Budkov O.	Done
	Create the wireframe and the diagram of screen transitions (#56)	Budkov O.	Done
	Prepare the report of Lab 2 (#57)	Budkov O.	Done

Table 3.3 – Sprint 3, 19.10–6.11.2026. Goal: a transparent and modular project (10 story points)

User story	Task	Assignee	Status
US-8: Track the work on GitHub with Scrum (#58) 5 SP, Must have	Create a GitHub account and a new repository (#59)	Budkov O.	Done
	Set up access to the repository (collaborators) (#60)	Budkov O.	Done
	Push the source code and reports to the repository (#61)	Budkov O.	Done
	Create the Product Backlog: user stories, labels and sprints (#62)	Budkov O.	Done
	Set up the Scrum board (#63)	Budkov O.	Done
	Prepare the report of Lab 3 (#64)	Budkov O.	Done

User story	Task	Assignee	Status
US-9: Split the program into modules (#65) 5 SP, Should have	Move the calculation and the checks into the core module (#66)	Budkov O.	Done
	Move console input, file loading and table printing into the io module (#67)	Budkov O.	Done
	Move the menu, result and help screens into the ui module (#68)	Budkov O.	Done
	Add build files: Makefile and CMakeLists.txt (#69)	Budkov O.	Done
	Check that the output is the same as before the split (#70)	Budkov O.	Done

Table 3.4 – Sprint 4, 9.11–27.11.2026. Goal: the requirements are specified (8 story points)

User story	Task	Assignee	Status
US-10: Agree on the requirements (#71) 8 SP, Must have	Choose a template for the SRS (#72)	Budkov O.	To do
	Write the purpose, definitions and system overview (#73)	Budkov O.	To do
	Write the overall description of the program (#74)	Budkov O.	To do
	Write the functional and non-functional requirements (#75)	Budkov O.	To do
	Prepare the report of Lab 4 (#76)	Budkov O.	To do

Table 3.5 – Sprint 5, 30.11–18.12.2026. Goal: a reliable and tested calculator (13 story points)

User story	Task	Assignee	Status
US-11: Never crash on bad input (#77) 5 SP, Must have	Add exception handling for wrong input and files (#78)	Budkov O.	To do
	Show one clear message for every type of error (#79)	Budkov O.	To do
US-12: Protect the results with unit tests (#80) 8 SP, Must have	Set up a unit test framework (#81)	Budkov O.	To do
	Write unit tests for the calculation of y (#82)	Budkov O.	To do
	Write unit tests for the input checks (#83)	Budkov O.	To do
	Prepare the report of Lab 5 (#84)	Budkov O.	To do

Table 3.6 – Product Backlog items that are not planned for a sprint yet

User story	Task	Assignee	Status
US-13: Choose the precision of the table (#85) 2 SP, Could have	not split into tasks yet	Budkov O.	To do
US-14: Export the table to CSV (#86) 3 SP, Could have	not split into tasks yet	Budkov O.	To do

Task 3.5 Working with the Git Repository

The source code and the reports were added to the repository from the command line in the folder of the project: git init, git add ., git commit -m "Add code and reports of Labs 1-3", git branch -M main, git remote add origin https://github.com/oleksiibudkov/SE-Lab-Var3.git and

git push -u origin main (Figures 3.13–3.14). After that, every change is made in small steps: the changed files are staged with git add, a commit with a short message is created with git commit and pushed to github.com with git push (Figures 3.15–3.16). A separate branch is used for a separate piece of work: the branch lab3-planning was created with git switch -c and pushed with git push -u origin lab3-planning (Figure 3.17).

The user story US-9 “Split the program into modules” was made in the branch feature/modular-structure. The single file main.cpp was split into three layers: core — the data, the calculation of y and the input checks without any input or output; io — console input, loading from a file and printing the table; ui — the menu, result and help screens. main.cpp keeps only the loop of the main menu. The program is built with a Makefile or with CMake (Visual Studio), and a regression test (make test) checks that the output is the same as before the split. Every commit names its task, for example “Move the screens into the ui module … Closes #68”, so GitHub closes the task when the branch is merged. The branch was merged into main through the pull request #87 (Figure 3.18); the history of the commits shows the merge (Figure 3.19).

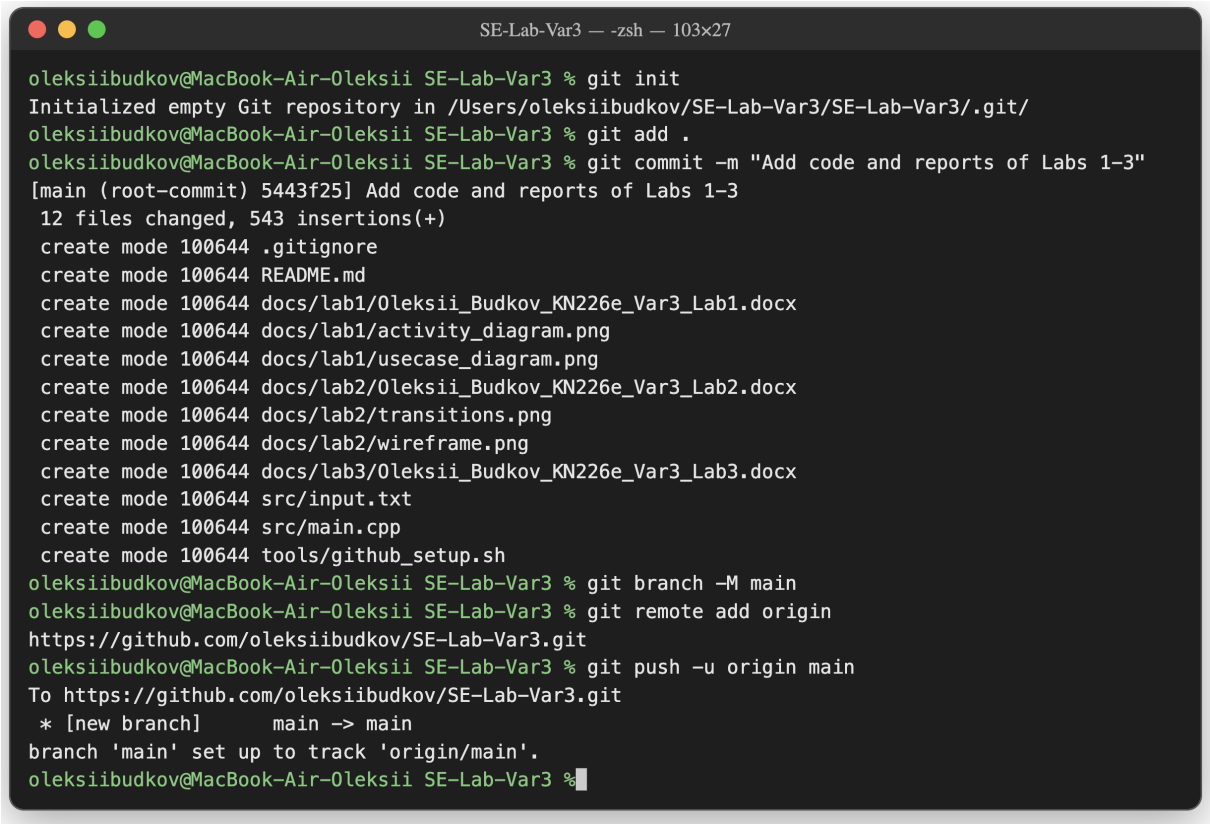


Figure 3.13 – Creating of a Git repository from the command line

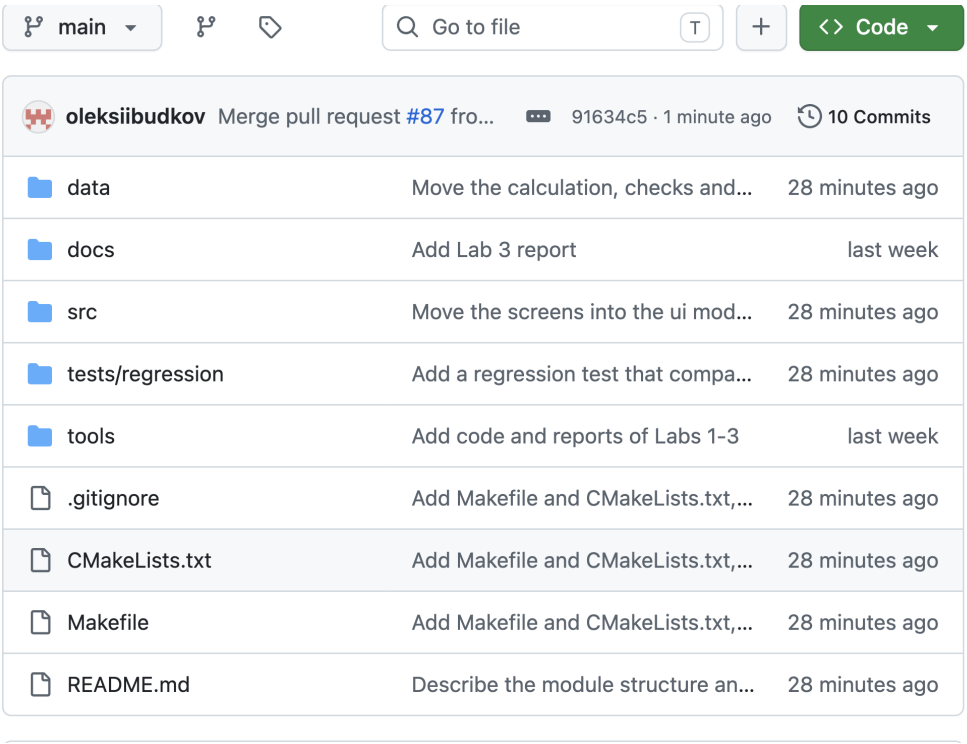


Figure 3.14 – Files of the project on github.com

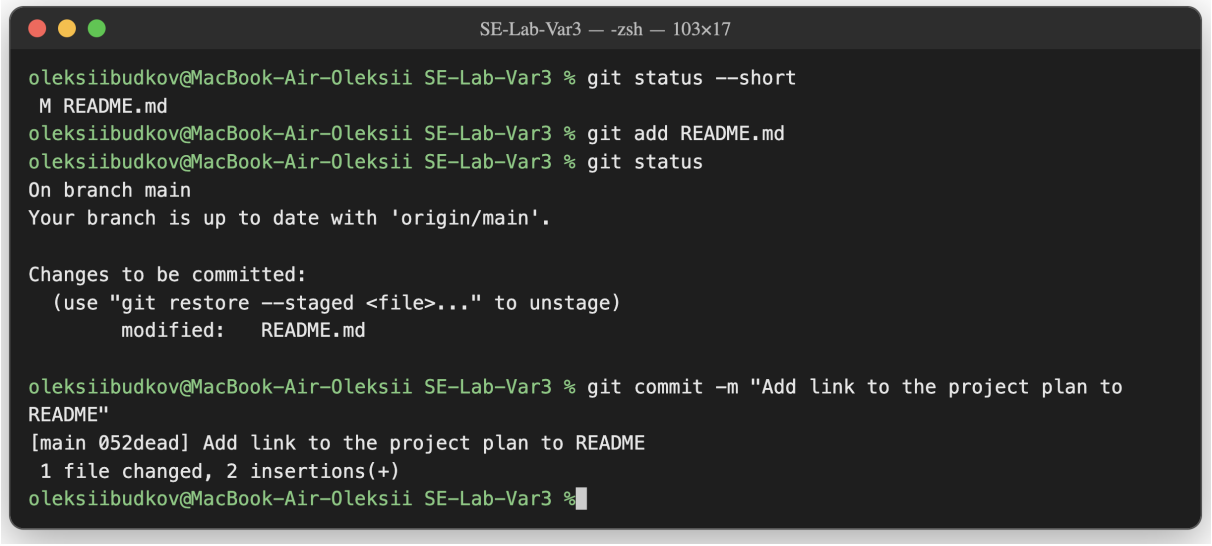


Figure 3.15 – Committing of the changed file



Figure 3.16 – Pushing of the commit

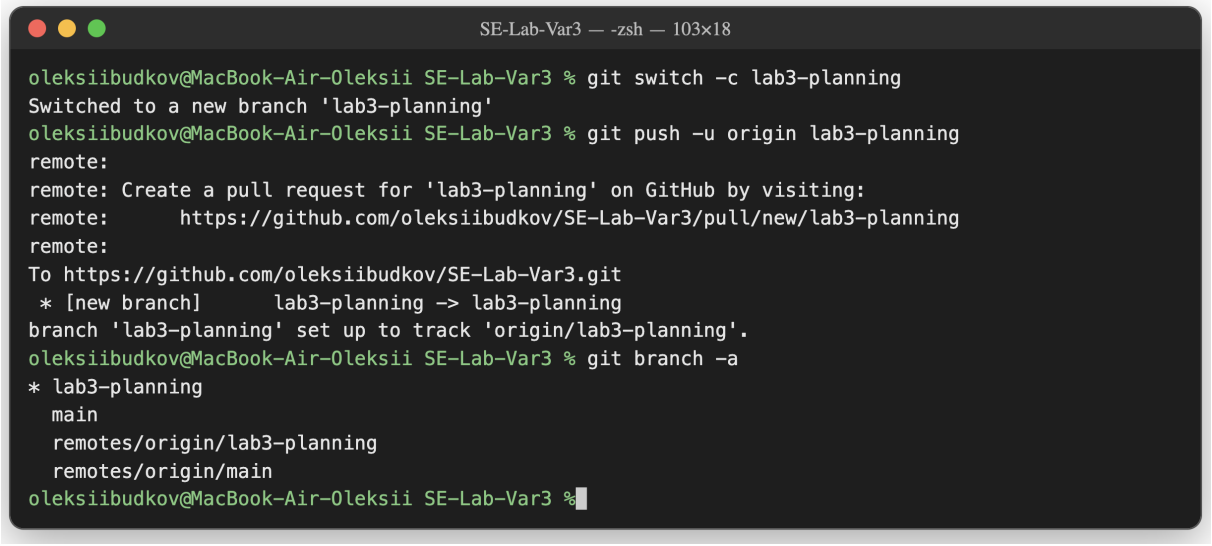


Figure 3.17 – Creating of a new branch

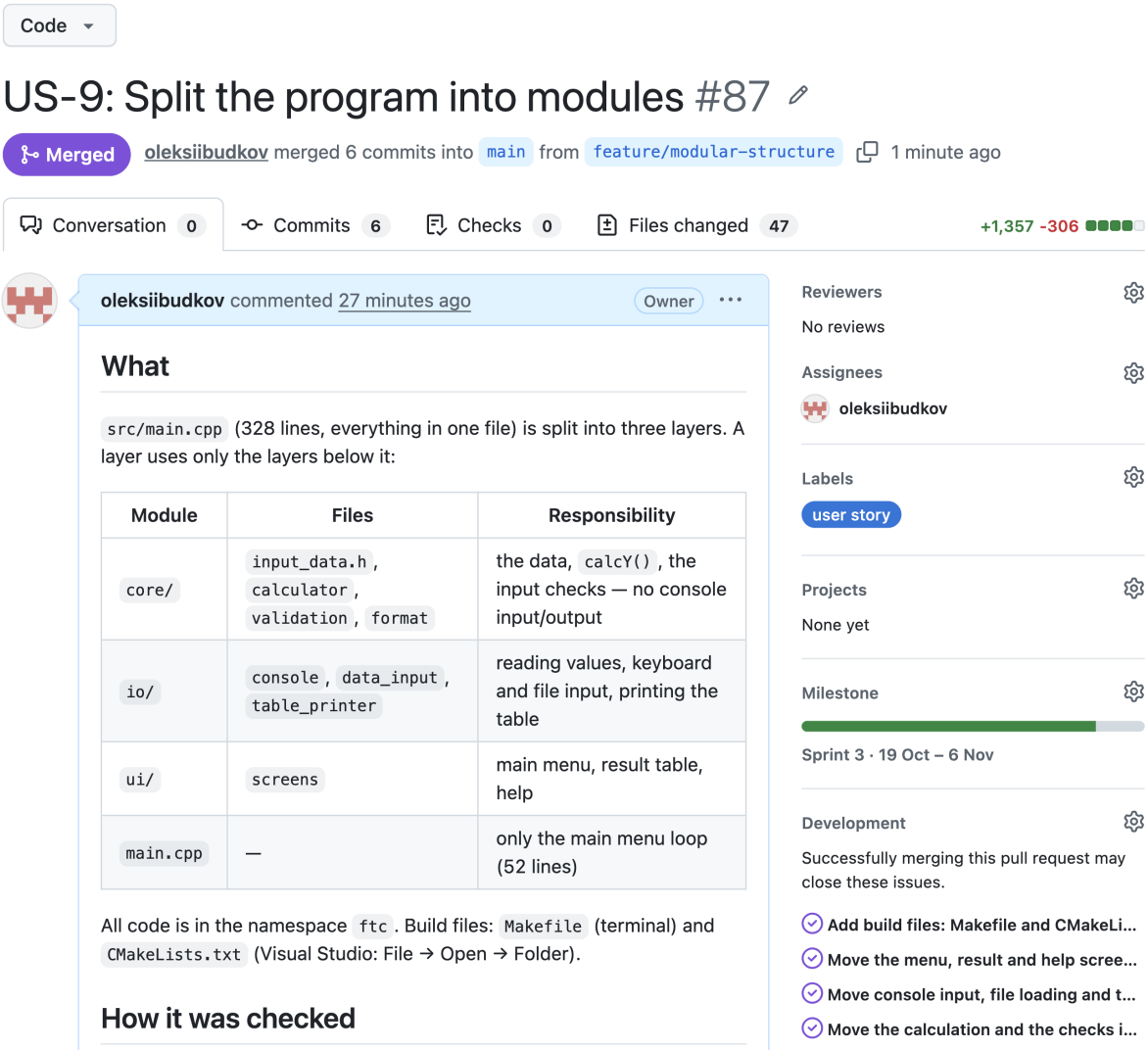


Figure 3.18 – Pull request of the user story US-9

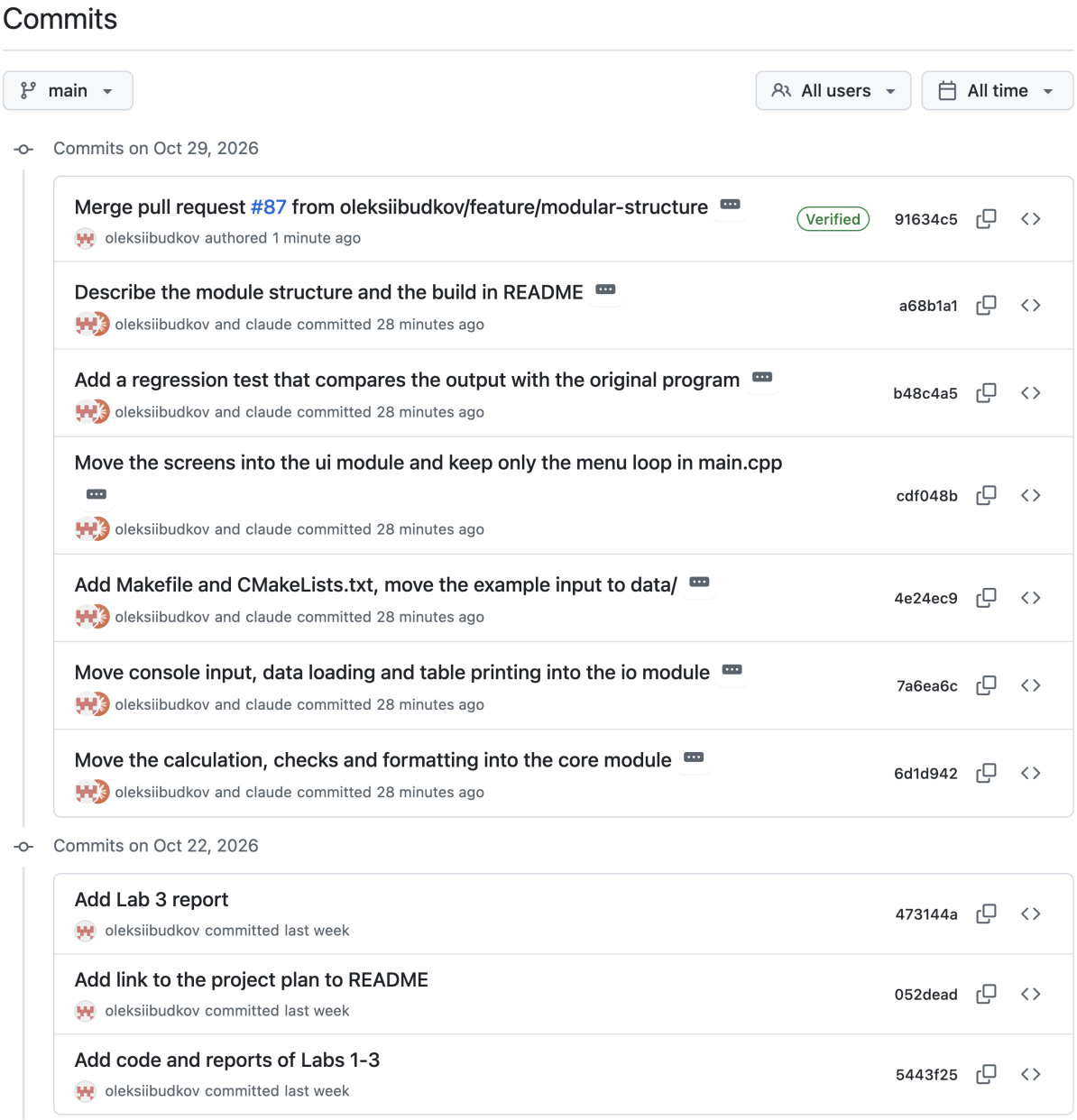


Figure 3.19 – History of commits

Conclusions

During this laboratory work we got skills and knowledge about working with the project tracking system GitHub. We registered an account, created a repository for the project and learned how to add collaborators to it. We planned the work of all laboratory works with Scrum: a Product Backlog of user stories with acceptance criteria, story points and MoSCoW priorities, five three-week sprints with sprint goals, tasks as sub-issues of their stories and a Scrum board with the columns Product Backlog, Sprint Backlog, In Progress and Done. We stored the code and the reports in the git repository from the command line, split the program into modules in a feature branch and merged it into the main branch through a pull request that closed the user story and its tasks automatically.

Appendix A. Main Tasks of the Project on GitHub

The main tasks of the project are the 14 user stories of the Product Backlog. Figures A.1–A.14 show the page of every story on GitHub: the description, the acceptance criteria, the estimate, the priority, the status on the Scrum board, the sprint and the tasks (sub-issues).

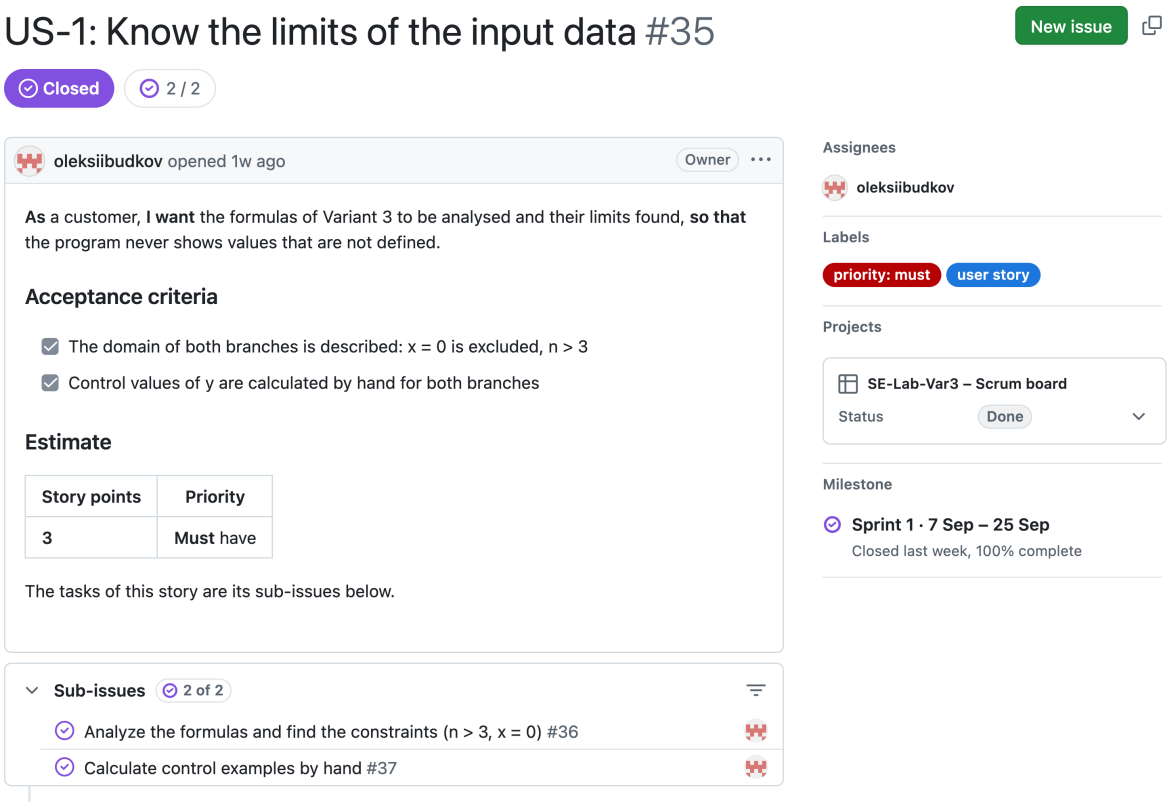


Figure A.1 – US-1: Know the limits of the input data (#35), Sprint 1, Done

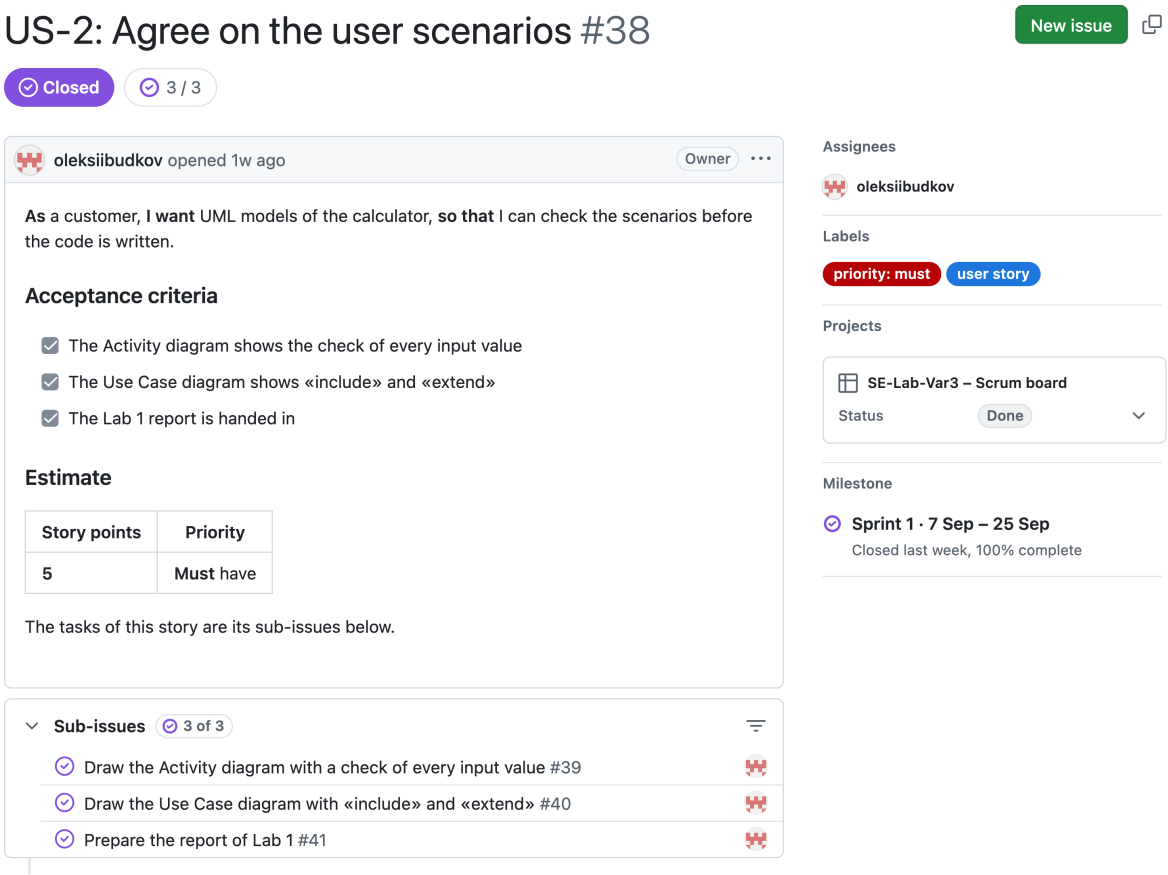


Figure A.2 – US-2: Agree on the user scenarios (#38), Sprint 1, Done

US-3: See the table of y(x) #42

New issue

Closed 3 / 3

oleksiibudkov opened 1w ago Owner

As a user, **I want** to see y for every x from the start to the end value with a chosen step, **so that** I can study how the function changes.

Acceptance criteria

- [x] Both branches (x < 3 and x ≥ 3) give the hand-calculated control values
- [x] For x = 0 the table shows "not defined"
- [x] Values that are too big are shown as "too large"

Estimate

Story points	Priority
5	**Must** have

The tasks of this story are its sub-issues below.

Sub-issues 3 of 3

- Implement the calculation of y for both branches #43
- Print the result table with “not defined” and “too large” #44
- Compare the results with the control examples #45

Assignees: oleksiibudkov

Labels: priority: must, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Done

Milestone: Sprint 2 · 28 Sep – 16 Oct — Closed last week, 100% complete

Figure A.3 – US-3: See the table of y(x) (#42), Sprint 2, Done

US-4: Enter data from the keyboard #46

New issue

Closed 2 / 2

oleksiibudkov opened 1w ago Owner

As a user, **I want** to type the input values and get a clear message about every wrong value, **so that** I can fix a mistake at once.

Acceptance criteria

- [x] Every value is checked right after it is typed
- [x] The message says what is wrong and gives an example
- [x] Text instead of a number does not break the program

Estimate

Story points	Priority
5	**Must** have

The tasks of this story are its sub-issues below.

Sub-issues 2 of 2

- Implement keyboard input with a separate check of every value #47
- Write clear error messages with examples #48

Assignees: oleksiibudkov

Labels: priority: must, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Done

Milestone: Sprint 2 · 28 Sep – 16 Oct — Closed last week, 100% complete

Figure A.4 – US-4: Enter data from the keyboard (#46), Sprint 2, Done

US-5: Load data from a file #49

New issue

Closed 2 / 2

oleksiibudkov opened 1w ago — Owner

As a user, **I want** to load the input values from a file, **so that** I don't type them every time.

Acceptance criteria

- [x] The file holds 4 numbers: start end step n
- [x] A wrong or missing file does not change the current data

Estimate

Story points	Priority
3	**Should** have

The tasks of this story are its sub-issues below.

Sub-issues 2 of 2

- Implement loading of data from a file #50
- Keep the previous data when the file is wrong #51

Assignees: oleksiibudkov

Labels: priority: should, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Done

Milestone: Sprint 2 · 28 Sep – 16 Oct — Closed last week, 100% complete

Figure A.5 – US-5: Load data from a file (#49), Sprint 2, Done

US-6: Save the table to a file #52

New issue

Closed 1 / 1

oleksiibudkov opened 1w ago — Owner

As a user, **I want** to save the result table to a file, **so that** I can use it later.

Acceptance criteria

- [x] After the table the program asks whether to save it
- [x] results.txt contains the data line and the same table

Estimate

Story points	Priority
2	**Should** have

The tasks of this story are its sub-issues below.

Sub-issues 1 of 1

- Implement saving of the table to results.txt #53

Assignees: oleksiibudkov

Labels: priority: should, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Done

Milestone: Sprint 2 · 28 Sep – 16 Oct — Closed last week, 100% complete

Figure A.6 – US-6: Save the table to a file (#52), Sprint 2, Done

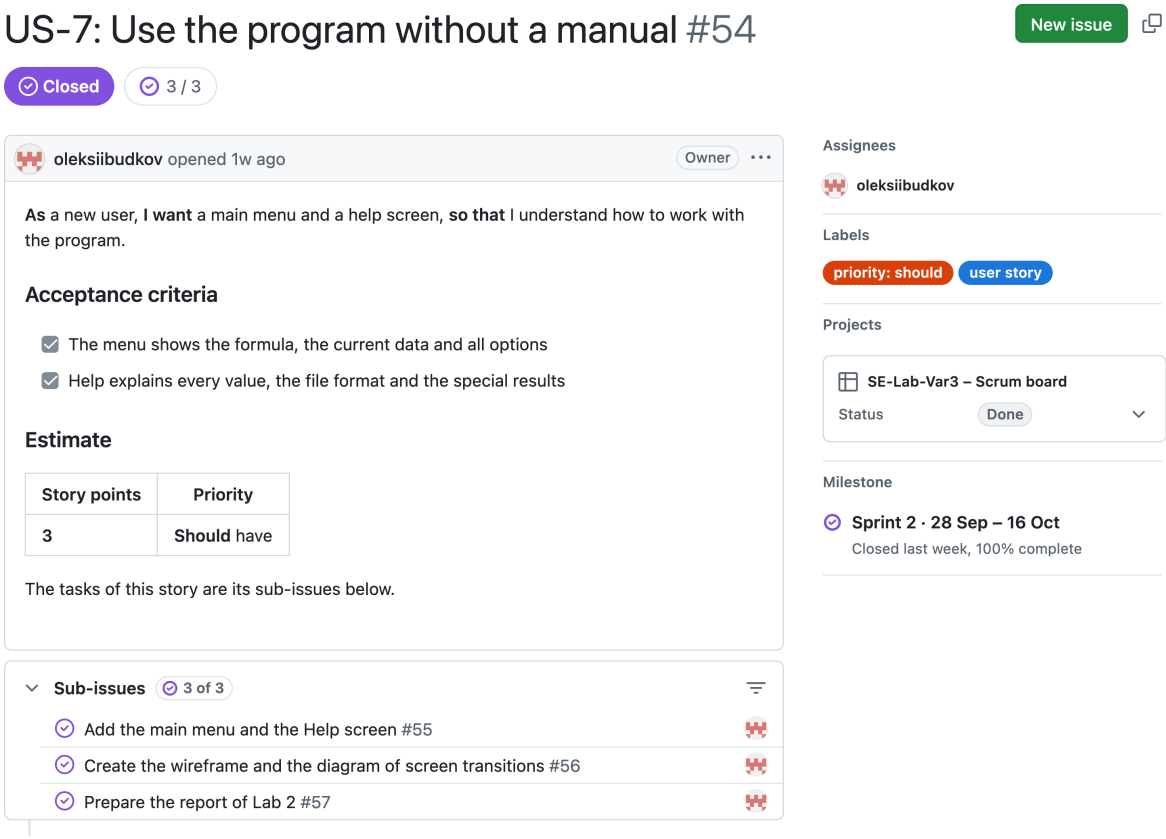


Figure A.7 – US-7: Use the program without a manual (#54), Sprint 2, Done

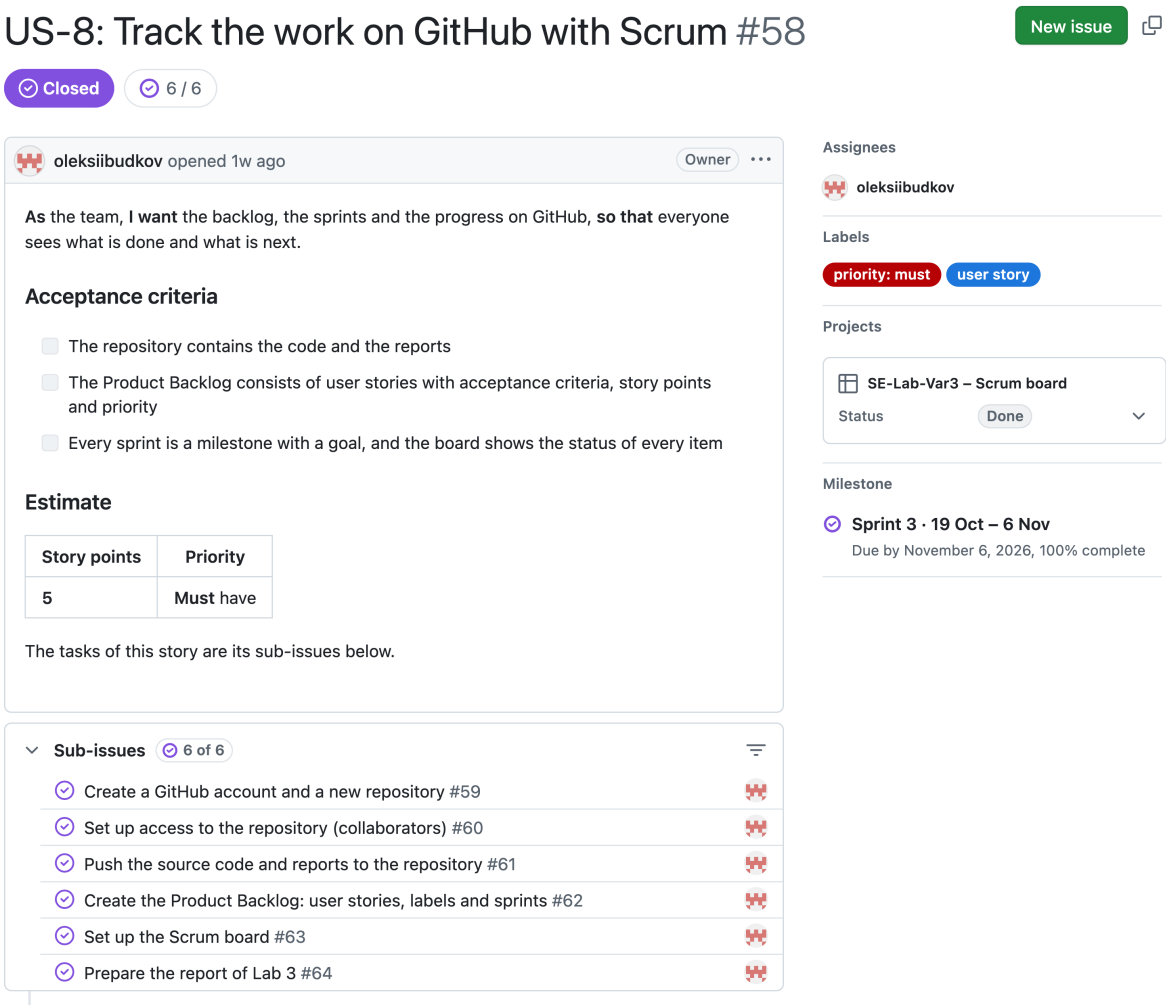


Figure A.8 – US-8: Track the work on GitHub with Scrum (#58), Sprint 3, Done

US-9: Split the program into modules #65

New issue

Closed · 5 / 5 · #87

oleksiibudkov opened 1w ago — Owner

As a developer, **I want** the program split into modules, **so that** each part can be read, changed and tested separately.

Acceptance criteria

- [ ] The calculation, the checks, the input/output and the screens are separate modules with headers
- [ ] The program behaves exactly as before: the same output for the same input
- [ ] It builds with make and with CMake (Visual Studio) without warnings

Estimate

Story points	Priority
5	**Should** have

The tasks of this story are its sub-issues below.

Sub-issues 5 of 5

- Move the calculation and the checks into the core module #66 — 1
- Move console input, file loading and table printing into the io module #67 — 1
- Move the menu, result and help screens into the ui module #68 — 1
- Add build files: Makefile and CMakeLists.txt #69 — 1
- Check that the output is the same as before the split #70 — 1

Assignees: oleksiibudkov

Labels: priority: should, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Done

Milestone: Sprint 3 · 19 Oct – 6 Nov
Due by November 6, 2026, 100% complete

Figure A.9 – US-9: Split the program into modules (#65), Sprint 3, Done

US-10: Agree on the requirements #71

New issue

Open 0 / 5

oleksiibudkov opened 1w ago — Owner

As a customer, **I want** a Software Requirements Specification of the calculator, **so that** I can accept the product against clear requirements.

Acceptance criteria

- [ ] The SRS follows a chosen template (for example IEEE 830)
- [ ] It has the purpose, definitions, overall description, functional and non-functional requirements
- [ ] Every requirement can be checked

Estimate

Story points	Priority
8	**Must** have

The tasks of this story are its sub-issues below.

Sub-issues 0 of 5

- Choose a template for the SRS #72
- Write the purpose, definitions and system overview #73
- Write the overall description of the program #74
- Write the functional and non-functional requirements #75
- Prepare the report of Lab 4 #76

Assignees: oleksiibudkov

Labels: priority: must, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Product Backlog

Milestone: Sprint 4 · 9 Nov – 27 Nov — Due by November 27, 2026

Figure A.10 – US-10: Agree on the requirements (#71), Sprint 4, Product Backlog

US-11: Never crash on bad input #77

New issue

Open 0 / 2

oleksiibudkov opened 1w ago — Owner

As a user, **I want** the program to handle any wrong input or file problem, **so that** it never stops unexpectedly.

Acceptance criteria

- [ ] Wrong values and file errors are reported with a message, not a crash
- [ ] Errors are handled with exceptions in one place

Estimate

Story points	Priority
5	**Must** have

The tasks of this story are its sub-issues below.

Sub-issues 0 of 2

- Add exception handling for wrong input and files #78
- Show one clear message for every type of error #79

Assignees: oleksiibudkov

Labels: priority: must, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Product Backlog

Milestone: Sprint 5 · 30 Nov – 18 Dec — Due by December 18, 2026

Figure A.11 – US-11: Never crash on bad input (#77), Sprint 5, Product Backlog

US-12: Protect the results with unit tests #80

New issue

Open 0 / 4

oleksiibudkov opened 1w ago Owner

As a developer, **I want** unit tests for the calculation and the input checks, **so that** a change cannot silently break the results.

Acceptance criteria

- [ ] Tests cover both branches of y(x), x = 0 and a big n
- [ ] Tests cover every input check
- [ ] All tests pass before a merge into main

Estimate

Story points	Priority
8	**Must** have

The tasks of this story are its sub-issues below.

Sub-issues 0 of 4

- Set up a unit test framework #81
- Write unit tests for the calculation of y #82
- Write unit tests for the input checks #83
- Prepare the report of Lab 5 #84

Assignees: oleksiibudkov

Labels: priority: must, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Product Backlog

Milestone: Sprint 5 · 30 Nov – 18 Dec — Due by December 18, 2026

Figure A.12 – US-12: Protect the results with unit tests (#80), Sprint 5, Product Backlog

US-13: Choose the precision of the table #85

New issue

Open

oleksiibudkov opened 1w ago Owner

As a user, **I want** to choose how many digits the table shows, **so that** the results are easier to read.

Acceptance criteria

- [ ] The precision from 1 to 10 digits is set in the menu
- [ ] It is used both on the screen and in results.txt

Estimate

Story points	Priority
2	**Could** have

The tasks of this story are its sub-issues below.

Assignees: oleksiibudkov

Labels: priority: could, user story

Projects: SE-Lab-Var3 – Scrum board — Status: Product Backlog

Milestone: No milestone

Figure A.13 – US-13: Choose the precision of the table (#85), not planned yet, Product Backlog

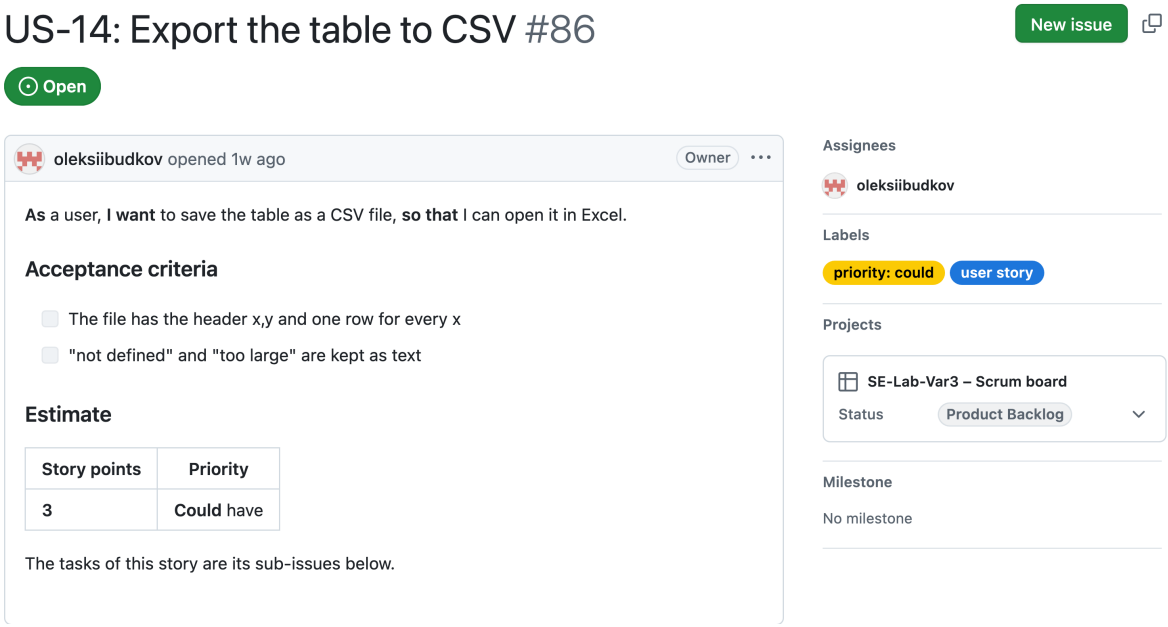


Figure A.14 – US-14: Export the table to CSV (#86), not planned yet, Product Backlog